

LADiM: LAYERED DIAGNOSIS FOR MULTI-AGENT CODE MIGRATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Migrating deep learning code across frameworks enables the reuse of training programs in new software and hardware environments. Framework differences can silently change gradients and parameter updates even when a translated program runs successfully. To address this problem, we propose LaDiM, a framework with multiple agents that uses dependencies between training computations to guide diagnosis and repair. The Verifier Agent investigates discrepancies and passes supporting evidence to a separate Repair Agent. The Orchestrator schedules their work and returns verification results after each submission. Repository context management preserves relevant evidence to coordinate repair across dependent files. On 50 migrations from PyTorch to MindSpore, LaDiM matches the state of the art in acceptance with 57.4% fewer total tokens, including initial translation. Evaluation across framework migration, language migration, and repository tasks demonstrates LaDiM’s effectiveness in preserving training behavior.

1 INTRODUCTION

Migrating deep learning code allows existing models and training programs to run in new software and hardware environments. Differences in operator semantics, numerical precision, and random number handling can change the computations performed after migration. Automatic differentiation and optimizer interfaces introduce further differences in how a model learns (Paszke et al., 2019; MindSpore, 2020). Large language models (LLMs) reduce the manual effort involved in code translation (He et al., 2025; Macedo et al., 2025b), while automated tests and interactive repair help assess and correct translated behavior (Rozière et al., 2022; Yang et al., 2024; Ibrahimzada et al., 2026). Training programs require these capabilities to preserve the computations that determine each parameter update.

A translated program can produce a loss close to that of the source while computing incorrect gradients. An optimizer error can also alter the parameter update without changing the current loss or gradients. We use semantic drift to describe these changes in training behavior caused by migration. Training computations have dependencies that help investigate such errors. A forward error may change gradients and updates, while an incorrect update may affect later forward computations. Locating the cause requires examining how the translated code produces the observed differences. This investigation also affects migration cost. Figure 1 compares complete token use on common migration tasks, where LaDiM achieves full acceptance with less than half the tokens used by MatchFixAgent.

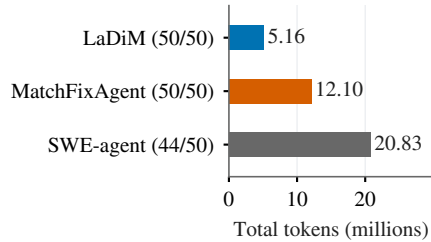


Figure 1: MindSpore migration costs, including initial translation and all subsequent calls. Labels give accepted tasks out of 50.

We introduce LaDiM (Layered Diagnosis for Multi-Agent Code Migration), which uses training behavior to guide diagnosis and repair of an initial translation. Its Verifier Agent uses Layered Diagnosis to connect training discrepancies to code inspections and tests, establishing where source and target computations diverge. For example, matching forward values with missing gradients directs

attention to parameter registration and the differentiation path. Independent evidence handoff supplies the supporting code observations, measurements, and hypotheses to a separate Repair Agent. The Orchestrator schedules their work and returns verification results after each submission. The Repair Agent retains its earlier attempts and feedback as repair history, connecting each correction to its measured effects and using unresolved discrepancies to guide the next attempt.

For repositories, migration must preserve shared implementations across multiple entry points. A change that corrects one caller can alter the behavior of another, and investigation may need evidence collected earlier in a different file. LaDiM’s repository context management combines Repository Structural Analysis, which exposes files and their interfaces, with Repair Dependency Graph Planning, which organizes related files into work units and coordinates their prerequisites. Notebook editing tools, evidence associated with code versions, and context reconstruction carry the investigation across files. Changes to shared code invalidate affected checks and prompt the agent to verify dependent callers again.

We evaluate migration acceptance and token use from common initial translations under a shared protocol. LaDiM completes all 50 migrations from PyTorch to MindSpore with 57.4% fewer tokens than MatchFixAgent, covering both programs that require repair and programs that pass the initial checks. It also accepts 9/18 migrations from Java/DJL to Python/PyTorch, compared with 8/18 for SWE-agent and MatchFixAgent. On the time series repository, all three agents complete migration, and LaDiM uses 68.1% and 80.8% fewer tokens than the two baselines. On recommendation, LaDiM achieves the highest behavioral coverage and passes all ten original tests. The JAX study verifies the same repair procedure on a second target framework. Our contributions are as follows.

- We develop Layered Diagnosis, which uses dependencies among execution, forward values, gradients, and parameter updates to identify the computations to investigate. Adding these signals increases full acceptance from 4/16 to 16/16 in the controlled repair study.
- We introduce independent evidence handoff and repair history to connect investigation, edits, and verification, together with repository context management for shared code. LaDiM repairs four of five failed saved translations, including errors in a supporting library. All three repair baselines repair none.
- We establish migration acceptance and efficiency under common evaluation protocols across frameworks, programming languages, and repositories. LaDiM preserves complete MindSpore acceptance with 57.4% fewer tokens than MatchFixAgent and reduces token use in the Java/DJL and time series comparisons.

2 RELATED WORK

Code Translation and Framework Migration. Neural translation methods learn code mappings from multilingual corpora and use automated tests or compiler representations to improve translations (Lachaux et al., 2020; Rozière et al., 2022; Szafraniec et al., 2023). ExeCoder encodes functional semantics, syntax structure, and variable dependencies (He et al., 2025), while InterTrans explores intermediate translation paths (Macedo et al., 2025b). Framework migration tools such as MindConverter supply mappings between programming interfaces (MindSpore, 2023). RepoTransAgent uses repository context, test failures, and reflection to refine translated code (Guan et al., 2025). LaDiM investigates how a translated training program computes and updates its state. At repository scale, it coordinates changes to shared interfaces and checks their effect across declared entry points.

Differential Testing and Deep Learning Verification. Differential testing exposes compiler and program faults through disagreement between related executions (Yang et al., 2011; Le et al., 2014). DeepXplore and DeepGauge investigate behavioral differences and test coverage in deep learning systems (Pei et al., 2017; Ma et al., 2018). More directly related to framework migration, TensorScope tests corresponding APIs across frameworks to identify inconsistent behavior (Deng et al., 2023). NblaFuzz compares execution scenarios to test automatic differentiation, including first-order and higher-order gradients (Yang et al., 2023). LaDiM uses training differences as evidence for code repair. Its Verifier Agent relates the measured forward values, gradients, and parameter updates to the implementation, develops hypotheses about their origin, and supplies evidence that the Repair Agent can test when selecting a change.

Automated Program Repair and LLM Agents. Automated program repair uses tests or semantic constraints to guide patch generation (Nguyen et al., 2013), and pretrained language models generate patches from code and failure evidence (Xia et al., 2023). Interactive agents combine repository navigation, editing, and execution (Yang et al., 2024; Shinn et al., 2023; Chen et al., 2024b; Wang et al., 2024). Multi-agent systems distribute development activities among specialized roles (Qian et al., 2024; Hong et al., 2024; Wu et al., 2024; Chen et al., 2024a). MatchFixAgent takes a source program and an existing translation and uses several semantic analyses to guide test generation and repair (Ibrahimzada et al., 2026). LaDiM includes initial translation and organizes diagnosis around dependencies in training computations. The independent investigation and its supporting observations pass to a separate Repair Agent, which uses measurements after each submission to revise its hypotheses. The comparison evaluates both methods from the same initial translation and includes translation costs.

3 METHODS

LaDiM uses dependencies between training computations to guide diagnosis and repair. The Translator first produces a candidate in the requested framework. The Verifier Agent investigates discrepancies between source and target executions and passes its findings and supporting observations to a separate Repair Agent. The Repair Agent tests these hypotheses and revises the candidate. The Orchestrator schedules their work and returns verification results after each submission to guide further repair. Figure 2 shows this process.

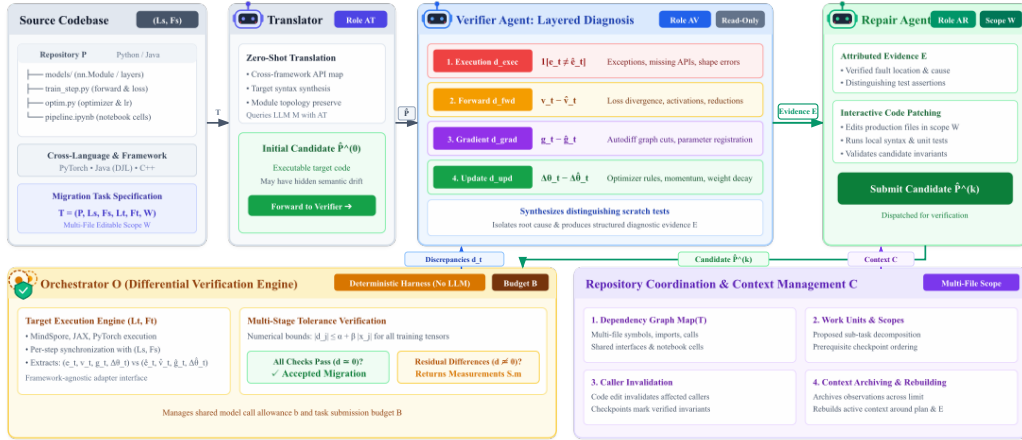


Figure 2: LaDiM’s migration workflow. The Translator generates a candidate, the Verifier Agent investigates code and training observations, and the Repair Agent receives the evidence and revises the candidate. The Orchestrator schedules their work, checks submissions, and returns measurements. Execution adapters obtain observations from MindSpore and JAX.

3.1 PRELIMINARY

A migration task $\mathcal{T} = (P, L_s, F_s, L_t, F_t, W)$ specifies a source program P , its language and framework (L_s, F_s) , the requested target environment (L_t, F_t) , and an editable file scope W . Migration produces a target program $\hat{P}$ that preserves the source’s training behavior. The scope can include the translated program, its supporting library, or a repository. An edit modifies code within W .

At step t , the source produces $(e_t, v_t, g_t, \theta_{t+1} - \theta_t)$, and the target produces $(\hat{e}_t, \hat{v}_t, \hat{g}_t, \hat{\theta}_{t+1} - \hat{\theta}_t)$. Here e_t records the execution outcome, including success or an exception, v_t contains selected forward values and losses, g_t contains gradients, and θ_t denotes the parameters before the update. After aligning corresponding tensors and parameters, we define

$$d_{\text{execution},t} = \mathbf{1}[e_t \neq \hat{e}_t], \quad d_{\text{forward},t} = v_t - \hat{v}_t, \quad (1)$$

$$d_{\text{gradient},t} = g_t - \hat{g}_t, \quad d_{\text{update},t} = (\theta_{t+1} - \theta_t) - (\hat{\theta}_{t+1} - \hat{\theta}_t). \quad (2)$$

For a numerical comparison $d = x - \hat{x}$, $d \simeq 0$ denotes agreement within its tolerance, such as $|d_j| \leq \alpha + \beta|x_j|$ for absolute tolerance α and relative tolerance β , or a specified norm bound. These discrepancies follow the training computation. Forward values determine the loss, differentiation produces the gradients, and the optimizer uses those gradients and its state to update parameters. The updated parameters then affect subsequent forward computations.

The LLM M performs translation, investigation, and repair under role instructions A_T , A_V , and A_R . The Orchestrator $\mathcal{O}$ evaluates candidates and manages their shared investigation and repair budget B . In the algorithms, S holds the candidate and observations, q is its verification status, H is a conversation, and C manages retained evidence. A submission sends the candidate for external verification after a repair stage that can contain several LLM calls, tests, and edits. Each $\text{QUERY}(M, A, H, b, B)$ invokes M with instructions A and conversation H , charging its usage to the stage allowance b and shared budget B .

3.2 VERIFIER AGENT

The Verifier Agent uses Layered Diagnosis to focus its investigation on the computations responsible for observed discrepancies. For an **execution discrepancy** ($d_{\text{execution},t} = 1$), it inspects the exception and the calls that produced it. For a **forward discrepancy** ($d_{\text{forward},t} \neq 0$), it traces the operators that produce different values and tests the suspected computation. When forward values agree but a **gradient discrepancy** ($d_{\text{gradient},t} \neq 0$) remains, it examines parameter registration and the path from the parameters to the loss. When both forward values and gradients agree, it investigates an **update discrepancy** ($d_{\text{update},t} \neq 0$) through the optimizer’s rule and state. The first affected step further guides this investigation because a changed update can alter later forward computations. The LLM selects code inspections and tests that distinguish possible causes, then records the code locations and observations supporting its hypotheses. Independent evidence handoff transfers these findings to a separate Repair Agent for assessment and correction. Algorithm 1 connects the observed discrepancies to this investigation and defines the shared `AGENTSTEP` for selecting actions, executing tools, and retaining evidence.

Algorithm 1 Layered Diagnosis

Require: Task $\mathcal{T}$, candidate and measurements S , LLM M , budget B , context C

Ensure: Evidence E : code observations, test results, locations, and hypotheses

Dependencies guiding the LLM’s investigation:
 $d_{\text{execution},t} = 1$: exception and failing calls
 $d_{\text{forward},t} \neq 0$: operators producing different values
 $d_{\text{forward},t} \simeq 0$, $d_{\text{gradient},t} \neq 0$: registration and differentiation
 $d_{\text{forward},t} \simeq 0$, $d_{\text{gradient},t} \simeq 0$, $d_{\text{update},t} \neq 0$: optimizer rule and state

- 1: **function** LAYEREDIAGNOSIS($\mathcal{T}, S, M, B, C$)
- 2: $H \leftarrow \text{STARTCONVERSATION}(A_V, \mathcal{T}, S)$; $b \leftarrow \text{DIAGNOSISALLOWANCE}(B)$
- 3: **while** WITHINLIMITS(b, B) **do**
- 4: $(H, a, R) \leftarrow \text{AGENTSTEP}(M, A_V, H, S, b, B, C)$
- 5: **if** $a.\text{complete}$ **then break**
- 6: $E \leftarrow \text{PUBLICEVIDENCE}(H, C.\text{archive})$
- 7: **return** E
- 8: **function** AGENTSTEP(M, A, H, S, b, B, C)
- 9: $H \leftarrow C.\text{BEFORECALL}(H, S)$
- 10: $a \leftarrow \text{QUERY}(M, A, H, b, B)$ ▷ Infer causes and select inspections, tests, or edits
- 11: $H \leftarrow H \parallel a$; $R \leftarrow []$
- 12: **for all** $c \in a.\text{toolCalls}$ **do**
- 13: $r \leftarrow \text{EXECUTE}(c, S, \text{productionReadOnly} = (A = A_V))$
- 14: $H \leftarrow H \parallel (c, r)$; $R \leftarrow R \parallel (c, r)$
- 15: $C.\text{RECORD}(c, r, S)$ ▷ Retain observations for subsequent reasoning
- 16: **return** (H, a, R)

3.3 REPAIR AGENT

The Repair Agent receives the Verifier Agent’s code observations, test results, and hypotheses through independent evidence handoff. It reassesses this evidence, runs further tests, and edits files in W , including supporting library code when the task permits it. The agent tests its changes before submission. The Orchestrator then verifies the candidate and returns the remaining discrepancies, which guide further investigation and repair.

Repair history preserves earlier edits, observations, and hypotheses across attempts. The continuing conversation includes failed tests and verification feedback, allowing the agent to relate a changed result to the preceding modification and revise explanations that did not resolve the fault. The current candidate also persists, so later attempts build on applied changes. Algorithm 2 reuses AGENTSTEP from Algorithm 1.

Algorithm 2 REPAIR($\mathcal{T}, S, E, q, M, B, C$)

```

1:  $H \leftarrow \text{STARTCONVERSATION}(A_R, \mathcal{T}, \text{AttributeToVerifier}(E))$ 
2:  $C.\text{handoff} \leftarrow \text{true}$ 
3: while SUBMISSIONSLEFT( $B$ )  $\wedge$  BUDGETLEFT( $B$ ) do
4:    $H \leftarrow H \parallel S.m; F \leftarrow (\text{false}, \text{false}); b \leftarrow \text{REPAIRALLOWANCE}(B)$ 
5:   while WITHINLIMITS( $b, B$ ) do
6:      $(H, a, R) \leftarrow \text{AGENTSTEP}(M, A_R, H, S, b, B, C); F \leftarrow \text{UPDATEFLAGS}(F, R)$ 
7:     if  $a.\text{complete} \wedge F.\text{edited} \wedge F.\text{tested}$  then break
8:     if  $a.\text{complete}$  then  $H \leftarrow H \parallel \text{PENDINGACTION}(F)$ 
9:      $(S.m, q) \leftarrow \mathcal{O}.\text{VERIFY}(S.\hat{P}); B.\text{submissions} \leftarrow B.\text{submissions} + 1$ 
10:    if  $q = \text{accepted}$  then
11:      return  $(S.\hat{P}, q)$ 
12: return  $(S.\hat{P}, q)$ 

```

3.4 REPOSITORY COORDINATION

A change to a shared implementation can affect several callers in a repository. Repository context management carries the plan, code context, and evidence needed to investigate these relationships across files. It combines Repository Structural Analysis, Repair Dependency Graph Planning, notebook tools with edit validation, an archive of evidence associated with code versions, and context reconstruction.

Repository Structural Analysis exposes files, imports, function and class definitions, and notebook cells for the agent to inspect through a tool call. The agent uses Repair Dependency Graph Planning to group related files into work units, specify their repair goals, and organize their prerequisites as a directed acyclic graph. It selects a unit whose prerequisites have current checkpoints, then edits the files assigned to that unit. Local syntax checks and selected tests establish each unit’s current status. When an edit changes a shared implementation, LaDiM invalidates the checks for affected units and their dependents, prompting the agent to check the relevant callers again. The Orchestrator evaluates the complete repository, including its entry points and training computations, to determine acceptance.

Notebook tools let the agent inspect and revise code cells while checking that edits preserve valid syntax. The evidence archive retains code observations, measurements, hypotheses, and conversations with their code versions, and supports retrieval of relevant records. At independent evidence handoff, a switch between work units, or a context capacity boundary, LaDiM rebuilds the conversation around the current plan, findings, and measurements. It retains relevant code observations when the corresponding files remain unchanged. These operations extend the program repair history to investigations that span multiple files and conversations. Algorithm 3 combines repository verification with the diagnosis and repair procedures.

Algorithm 3 Repository migration and context operations

```

1: function REPOSITORYMIGRATE( $\mathcal{T}, M, B$ )
2:    $S.\hat{P} \leftarrow \text{TRANSLATE}(M, A_T, \mathcal{T}); C \leftarrow \text{REPOSITORYCONTEXT}(\mathcal{T})$ 
3:    $(S.m, q) \leftarrow \mathcal{O}.\text{VERIFY}(S.\hat{P})$ 
4:   if  $q = \text{accepted}$  then
5:     return  $(S.\hat{P}, q)$ 
6:    $E \leftarrow \text{LAYEREDIAGNOSIS}(\mathcal{T}, S, M, B, C)$ 
7:   return REPAIR( $\mathcal{T}, S, E, q, M, B, C$ )
8: function C.RECORD( $c, r, S$ )
9:    $C.\text{archive} \leftarrow C.\text{archive} \parallel (c, r)$ 
10:  if CODECHANGED( $r$ ) then
11:    MARKHISTORICAL( $C.\text{measurements}$ )
12:    INVALIDATE( $C.\text{checkpoints}, \text{AffectedUnitsAndDependents}(r, C.\text{plan})$ )
13:     $(C.\text{plan}, C.\text{unit}, C.\text{checkpoints}) \leftarrow \text{UPDATE}(c, r, C)$ 
14: function C.BEFORECALL( $H, S$ )
15:  if  $C.\text{handoff} \vee \text{UNITSWITCHED}(C) \vee \text{ATCAPACITY}(H)$  then
16:    ARCHIVE( $C, H$ )
17:     $K \leftarrow \text{CURRENTREADCODE}(C.\text{unit}, S.\hat{P})$ 
18:     $E \leftarrow \text{PUBLICFINDINGS}(H, C.\text{archive})$ 
19:     $H \leftarrow \text{REBUILD}(H.\text{role}, C.\text{plan}, E, S.m, K)$ 
20:     $C.\text{handoff} \leftarrow \text{false}; C.\text{previousUnit} \leftarrow C.\text{unit}$ 
21:  return  $H$ 

```

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Tasks. The main comparison contains 50 migration tasks from PyTorch to MindSpore, covering tensor operations, CNNs, MLPs, Transformers, language models, and parameter and optimizer interfaces. We also migrate 18 Java/DJL programs to Python/PyTorch, comprising eight textbook examples and ten applications (DJL Contributors, 2026b;a), and two public repositories for time series (jinglescode, 2026) and recommendation (gauravchak, 2026). The repositories contain 13 and 24 code files or notebooks, totaling 865 and 2,086 nonblank lines. To examine repair of actual translation errors, we use ten saved initial translations: five require repair and five pass the initial checks. This study permits changes to the candidates and their supporting library. A separate study controls the training signals on 16 faulty candidates. The JAX study uses six candidates, with execution, forward, and gradient faults in an MLP and a CNN. A separate detection study follows four models over 50 training steps with three seeds to compare direct gradient and update checks with loss checks. Appendix A.1 summarizes the program collections in Table 5.

Compared methods. We compare LaDiM with SWE-agent (Yang et al., 2024) and MatchFix-Agent (Ibrahimzada et al., 2026) using a common DeepSeek backend and each baseline’s native algorithms, prompts, tools, parsers, and retry policies. The main table also includes Direct LLM initial translation, CodeTransEngine’s direct translation configuration (Macedo et al., 2025a), MSAdapter (OpenI, 2025), InterTrans (Macedo et al., 2025b), and a repair loop that rewrites the complete candidate from test feedback. Direct repair in the analysis studies is a separate control that receives the saved candidate and uses LaDiM’s inspection, testing, and editing tools with its existing repair procedure. Each table lists the methods evaluated in that study.

For JAX, Ivy converts code between frameworks by transpiling the supplied model (Ivy Contributors, 2026), while `torch2jax` maps PyTorch operations and tensors to JAX through native conversion (torch2jax Contributors, 2026). We apply each converter once to the supplied faulty candidates and measure whether the existing faults are removed, without LLM repair. TorchAX, a PyTorch frontend for JAX (Google, 2025), is the execution backend for LaDiM and Direct repair, with JAX differentiation and Optax parameter updates.

Table 1: Migration acceptance and token use on common inputs. Panels (a,b) compare individual programs, and panels (c,d) compare repositories. End-to-end costs include the shared initial translation once and all subsequent calls, including failed attempts. A dash denotes a method without LLM calls.

(a) PyTorch to MindSpore

(b) Java/DJL to Python/PyTorch

Method	LLM calls	Tokens (millions)	Accepted	Method	LLM calls	Tokens (millions)	Accepted
Direct LLM	29	0.429	29/50	Direct LLM	18	0.694	1/18
CodeTransEngine	29	0.280	31/50	InterTrans	89	2.286	1/18
MSAdapter	–	–	15/50	Test-guided repair	71	4.475	7/18
SWE-agent	1,097	20.831	44/50	SWE-agent	682	24.321	8/18
MatchFixAgent	619	12.097	50/50	MatchFixAgent	524	28.355	8/18
LaDiM (ours)	326	5.159	50/50	LaDiM (ours)	345	21.479	9/18

(c) Time series repository

(d) Recommendation repository

Method	Behavior checks passed	Entry points	Repair calls	Tokens (millions)	Behavior checks passed	Original tests	Repair calls	Tokens (millions)
SWE-agent	69/69	3/3	80	8.200	104/145	10/10	80	8.317
MatchFixAgent	69/69	3/3	69	13.598	129/145	0/10	80	8.679
LaDiM (ours)	69/69	3/3	35	2.614	130/145	10/10	80	8.811

MatchFixAgent passes 0/10 original recommendation tests because an unmigrated PyTorch import prevents test collection. Complete token costs and repository gradient and parameter update checks appear in Tables 6 and 9.

Evaluation Protocol. Methods use common source programs, evaluation inputs, and corresponding initial parameters. Repair methods also receive the same initial translations. Acceptance requires agreement in execution, forward values, gradients, and parameter updates, together with the task’s optimizer and interface checks. Accepted programs pass every required check. For repositories, *Behavior checks passed* counts successful execution, numerical, and interface checks. *Entry points* counts successful execution of the training script, notebook, and teaching examples. *Original tests* counts passing tests supplied by the repository. The main migration comparisons check consecutive training steps. Candidates that pass the public evaluation are checked on additional seeds. Investigation and repair allow up to 40 LLM calls per program and 80 per repository, with at most four submissions. Appendix A gives the numerical tolerances, seeds, and complete budgets.

4.2 MAIN RESULTS

Migration to MindSpore. LaDiM completes all 50 MindSpore migrations with 57.4% fewer tokens than MatchFixAgent (Table 1a). The savings cover both programs that require repair and programs that pass the initial checks (Figure 3). Reduced input usage accounts for 93.0% of the token savings, locating most of the difference in the context processed during investigation and repair. LaDiM accepts 46/50 tasks after the first submission and all 50 after the second, retaining full acceptance at four submissions.

Migration across languages. LaDiM accepts all programs completed by MatchFixAgent and an additional recurrent network example in the Java/DJL collection (Table 1b). Its 9/18 accepted programs comprise all eight textbook examples and one application, compared with 8/18 for SWE-agent and MatchFixAgent. LaDiM uses 11.7% fewer tokens than SWE-agent and 24.2% fewer than MatchFixAgent. These results extend training behavior repair to tasks that change both the programming language and framework.

Repository migration. All three agents complete the time series migration, preserving the shared model across a training script, notebook, and teaching examples. At the same complete acceptance of 69/69 checks, LaDiM uses 68.1% fewer end-to-end tokens than SWE-agent and 80.8% fewer than MatchFixAgent. The coordinated repair therefore preserves training behavior across several callers with substantially lower token use.

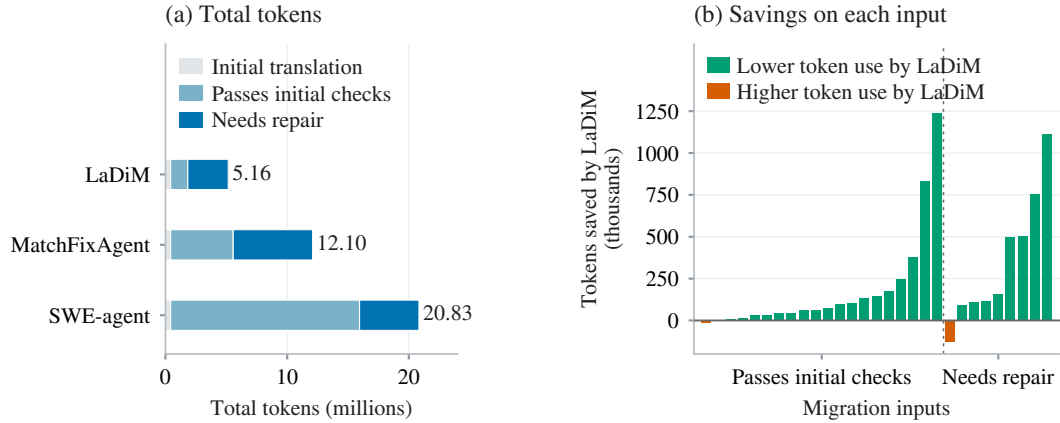


Figure 3: Token costs on MindSpore migration. (a) LaDiM, MatchFixAgent, and SWE-agent totals include initial translation and subsequent calls grouped by the initial checks. (b) Each bar shows MatchFixAgent tokens minus LaDiM tokens for one distinct source and evaluation input, sorted within each group. LaDiM and MatchFixAgent both accept 50/50 tasks. LaDiM uses 57.4% fewer tokens. Positive values indicate lower LaDiM cost.

On recommendation, LaDiM restores reward model inference and retrieval, which remain unresolved in both baselines, and executes the training command. It achieves the highest behavioral coverage, passing 130/145 checks and all ten original tests. These results cover the shared model’s use in training, inference, and retrieval. Appendix E gives the remaining numerical differences and detailed checks.

4.3 ANALYSIS EXPERIMENTS

Repairing Translation Errors. Table 2 reports failed programs repaired, passing programs preserved, and final accepted programs for the ten saved translations. LaDiM repairs four of the five failed programs and preserves all five passing programs, raising final acceptance from five to nine. Direct repair, SWE-agent, and MatchFixAgent retain the initial five successes but recover none of the failed programs. In the LSTM case, LaDiM repairs recurrent operations and tensor indexing in the supporting library, restoring agreement in loss, gradients, and parameter updates. This example shows how training discrepancies guide repair into library implementations used by the translated program (Appendix D).

Table 2: Repair and preservation on ten saved initial translations. Five programs require repair and five pass the initial checks. Costs include all investigation and repair calls, with candidates and supporting libraries editable.

Method	Failed programs repaired	Passing programs preserved	Repair tokens (millions)	Final accepted
Direct repair	0/5	5/5	3.751	5/10
SWE-agent	0/5	5/5	12.678	5/10
MatchFixAgent	0/5	5/5	14.351	5/10
LaDiM	4/5	5/5	13.207	9/10

Training Signals and Detection Latency. Gradient errors can leave the current loss unchanged. Errors in parameter updates can preserve both current forward values and gradients, with their effects appearing in later steps. LaDiM’s gradient and update checks detect their respective faults at step 1 in all 12 runs per fault, while the mean loss difference curves cross the detection threshold at steps 31 and 18 (Figure 4). The trajectories cover four models for 50 steps with source and target executions retaining separate state. Individual loss trajectories cross within this window in only 3/12 and 4/12 runs. Observing the affected training computation therefore exposes these faults before they produce

a detectable loss difference. Appendix A.6 gives the thresholds, synchronized checks, and healthy controls.

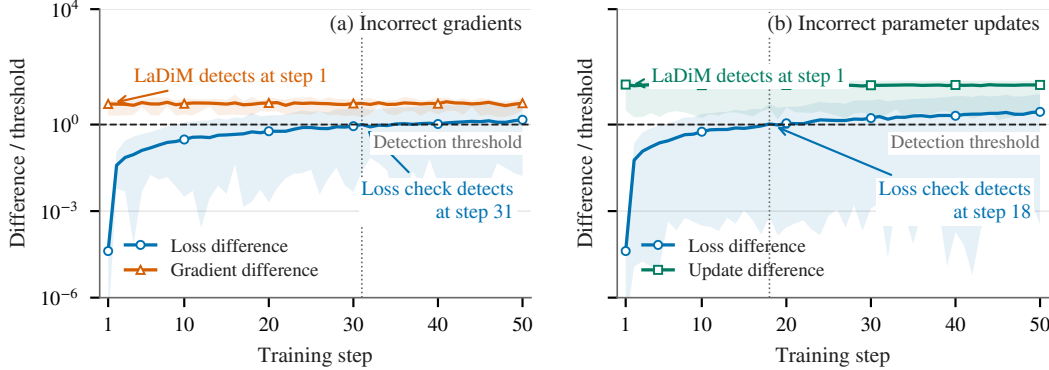


Figure 4: Training signal detection and loss threshold crossings. LaDiM detects gradient scaling (a) and partial update suppression (b) at step 1 in all 12 runs per panel. The loss check crosses its threshold at steps 31 and 18 on the mean curves. Individual loss trajectories cross within 50 steps in 3/12 and 4/12 runs. Differences are normalized by their thresholds, with curves showing means and shading showing ranges.

Generalization Across Frameworks. LaDiM repairs all six JAX cases spanning execution, forward, and gradient faults in MLP and CNN models (Table 3). Every case passes single-step SGD verification on three seeds after the first submission. Direct repair also accepts all six cases and uses fewer tokens. Ivy and `torch2jax` accept none of the faulty candidates after native conversion, while their healthy MLP and CNN controls pass. In these cases, native conversion preserves the supplied execution or numerical faults, and explicit repair restores agreement in training behavior. The same diagnosis and repair procedure thus works beyond MindSpore with JAX differentiation and parameter updates. Appendix A.5 details these controls; Appendix H reports the separate study of native JAX translations with consecutive training steps.

Table 3: Repair and native conversion controls on six faulty JAX candidates. LLM repair allows up to four submissions, with costs covering investigation and repair. Native conversion runs once without an LLM and is accepted only if the supplied fault is removed.

Metric	LLM repair		Single native conversion	
	LaDiM	Direct repair	Ivy	<code>torch2jax</code>
Accepted	6/6	6/6	0/6	0/6
LLM calls	76	57	0	0
LLM tokens	674,009	353,311	0	0

4.4 ABLATION STUDIES

We evaluate training signals on 16 faulty candidates, repair history and independent evidence hand-off on ten saved translations, and repository context management and Repository Structural Analysis on the time series repository. Table 4 reports acceptance and token use for these three studies.

Training signals. We compare cumulative feedback on 16 fixed candidates, with one execution, forward, gradient, and update fault in each CNN, image MLP, Transformer classifier, and small causal language model. Sources, candidates, tools, and budgets are fixed. Available feedback determines whether repair begins and stops. Each added signal exposes another fault class, increasing final acceptance under complete verification from 4/16 to 16/16 (Table 4a). Initially undetected faults decrease from 12 to 8 to 4 to 0 across the four settings. Forward values reveal changed computations, gradients reveal differentiation faults, and updates reveal optimizer faults.

Table 4: Effects of training signals and repair components. (a) Each + adds a signal to the preceding row, ending with all four signals on 16 faulty candidates. Final acceptance uses complete verification. (b) Ten saved translations: parentheses count failed programs repaired, and all conditions retain 5/5 initially passing programs. (c) Independent pairs on time series, with every condition passing 69/69 behavior checks on three seeds.

(a) Training signals			(b) Repair history and independent evidence handoff		
Feedback	Accepted	Repair tokens	Condition	Accepted (repaired)	Repair tokens
Execution and basic checks	4/16	2,088,915	Independent evidence handoff	9/10 (4/5)	15,547,813
+ Forward values	8/16	2,603,211	Continuous	8/10 (3/5)	14,516,463
+ Gradients	12/16	3,302,873	conversation		
+ Parameter updates	16/16	3,418,960	Without repair history	5/10 (0/5)	10,851,537

(c) Repository components on time series					
Component	Without component		With component		Token reduction
	Calls	Tokens	Calls	Tokens	
Repository context management	26	6,047,074	22	1,936,579	68.0%
Repository Structural Analysis	42	2,651,248	30	2,126,900	19.8%

Panels (a,b) include investigation in repair tokens. Panel (b) includes editing examples and feedback for malformed edits. In (c), calls cover investigation and repair, and tokens also include initial translation. Each condition runs once, with a separate complete reference for each pair.

Repair history and independent evidence handoff. Repair history and independent evidence handoff improve repair on the saved initial translations (Table 4b). With repair history, LaDiM repairs four of five failed programs. Removing that history reduces the count to zero. Independent evidence handoff repairs one more program than a continuous investigation and repair conversation, increasing final acceptance from 8/10 to 9/10. All three conditions preserve the five programs that pass the initial checks. The complete reference retains earlier attempts and their verification feedback while allowing the Repair Agent to reassess the Verifier’s evidence in its own conversation.

Repository context management and Repository Structural Analysis. On the time series repository, repository context management reduces end-to-end token use by 68.0% while preserving acceptance on all 69 checks (Table 4c). Both conditions retain investigation and independent evidence handoff. In a separate paired ablation, Repository Structural Analysis reduces end-to-end token use by 19.8% and LLM calls from 42 to 30, with the same complete acceptance. Structural context identifies relevant implementations, while the broader context mechanism carries plans, measurements, and inspected code across the investigation. Appendix B gives the complete program component results, and Appendix G reports both repositories’ cumulative comparisons and independent ablations of Repository Structural Analysis and Repair Dependency Graph Planning.

5 CONCLUSION

We introduced LaDiM, a framework with multiple agents for preserving training behavior during code migration. Layered Diagnosis uses dependencies among execution, forward values, gradients, and parameter updates to guide investigation. Independent evidence handoff and repair history connect the resulting evidence to corrective edits and their verification results. Repository context management carries this process across shared implementations and multiple entry points. LaDiM completes all 50 MindSpore migrations with 57.4% fewer tokens than MatchFixAgent, and the translation error study demonstrates repairs that all three repair baselines leave unresolved. Results across framework, language, and repository migration establish the effectiveness of this approach in preserving training behavior and reducing repair costs.

AI USE STATEMENT

REFERENCES

- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=EHg5GDnyq1>.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=KuPixIqPiq>.
- Zizhuang Deng, Guozhu Meng, Kai Chen, Tong Liu, Lu Xiang, and Chunyang Chen. Differential testing of cross deep learning framework APIs: Revealing inconsistencies and vulnerabilities. In *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 7393–7410. USENIX Association, 2023. URL <https://www.usenix.org/conference/usenixsecurity23/presentation/deng-zizhuang>.
- DJL Contributors. Dive into deep learning with java. <https://github.com/deepjavalibrary/d2l-java>, 2026a. Source repository, accessed September 21, 2026.
- DJL Contributors. Deep java library. <https://github.com/deepjavalibrary/djl>, 2026b. Source repository, accessed September 21, 2026.
- gauravchak. Two tower models. https://github.com/gauravchak/two_tower_models, 2026. Source repository, accessed September 21, 2026.
- Google. torchax: A PyTorch frontend for JAX. <https://google.github.io/torchax/>, 2025. Accessed: 2026-06-02.
- Ziqi Guan, Xin Yin, Zhiyuan Peng, and Chao Ni. RepoTransAgent: Multi-agent LLM framework for repository-aware code translation. *arXiv preprint arXiv:2508.17720*, 2025. doi: 10.48550/arXiv.2508.17720. URL <https://arxiv.org/abs/2508.17720>.
- Minghua He, Yue Chen, Fangkai Yang, Pu Zhao, Wenjie Yin, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. ExeCoder: Empowering large language models with executability representation for code translation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 7099–7125, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.362. URL <https://aclanthology.org/2025.emnlp-main.362/>.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- Ali Reza Ibrahimzada, Brandon Paulsen, Reyhaneh Jabbarvand, Joey Dodds, and Daniel Kroening. MatchFixAgent: Language-agnostic autonomous repository-level code translation validation and repair. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=MuyXpH3GL1>.
- Ivy Contributors. Ivy: Convert machine learning code between frameworks. <https://github.com/unifyai/ivy>, 2026. Accessed: 2026-09-22.
- jinglescode. Time series forecasting with pytorch. <https://github.com/jinglescode/time-series-forecasting-pytorch>, 2026. Source repository, accessed September 21, 2026.

- Marie-Anne Lachaux, Baptiste Rozière, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. In *Advances in Neural Information Processing Systems*, volume 33, pp. 20601–20611, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html>.
- Vu Le, Mehrdad Afshari, and Zhendong Su. Compiler validation via equivalence modulo inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 216–226. ACM, 2014. doi: 10.1145/2594291.2594334. URL <https://doi.org/10.1145/2594291.2594334>.
- Lei Ma, Felix Juefei-Xu, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Chunyang Chen, Ting Su, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. DeepGauge: Multi-granularity testing criteria for deep learning systems. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, pp. 120–131. ACM, 2018. doi: 10.1145/3238147.3238202. URL <https://doi.org/10.1145/3238147.3238202>.
- Marcos Macedo, Yuan Tian, and Bram Adams. CodeTransEngine: Ready-to-use backend for LLM-based code translation. In *ICLR 2025 Third Workshop on Deep Learning for Code*, 2025a. URL <https://openreview.net/forum?id=9WHJuzjPJf>.
- Marcos Macedo, Yuan Tian, Pengyu Nie, Filipe R. Cogo, and Bram Adams. InterTrans: Leveraging transitive intermediate translations to enhance LLM-based code translation. In *Proceedings of the 47th International Conference on Software Engineering*, pp. 2819–2831. IEEE, 2025b. doi: 10.1109/ICSE55347.2025.00236. URL <https://doi.org/10.1109/ICSE55347.2025.00236>.
- MindSpore. MindSpore: A new deep learning computing framework. <https://www.mindspore.cn/en>, 2020. Framework white paper and documentation.
- MindSpore. Migrating from third party frameworks with MindConverter. https://www.mindspore.cn/mindinsight/docs/en/r2.0.0-alpha/migrate_3rd_scripts_mindconverter.html, 2023. Accessed: 2026-06-02.
- Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. SemFix: Program repair via semantic analysis. In *Proceedings of the 35th International Conference on Software Engineering*, pp. 772–781. IEEE, 2013. doi: 10.1109/ICSE.2013.6606623. URL <https://doi.org/10.1109/ICSE.2013.6606623>.
- OpenI. MSAdapter. <https://openi.pcl.ac.cn/OpenI/MSAdapter/src/tag/v0.6.0>, 2025. Version 0.6.0. Accessed: 2026-09-20.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pp. 8024–8035, 2019. URL <https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>.
- Kexin Pei, Yinzhi Cao, Junfeng Yang, and Suman Jana. DeepXplore: Automated whitebox testing of deep learning systems. In *Proceedings of the 26th Symposium on Operating Systems Principles*, pp. 1–18. ACM, 2017. doi: 10.1145/3132747.3132785. URL <https://doi.org/10.1145/3132747.3132785>.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 15174–15186. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.810. URL <https://aclanthology.org/2024.acl-long.810/>.

- Baptiste Rozière, Jie M. Zhang, Francois Charton, Mark Harman, Gabriel Synnaeve, and Guillaume Lample. Leveraging automated unit tests for unsupervised code translation. In *Proceedings of the Tenth International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=cmt-6KtR4c4>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Marc Szafraniec, Baptiste Rozière, Hugh James Leather, Patrick Labatut, Francois Charton, and Gabriel Synnaeve. Code translation with compiler representations. In *Proceedings of the Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=XomEU3eNeSQ>.
- torch2jax Contributors. torch2jax: Run PyTorch in JAX. <https://github.com/samuella/torch2jax>, 2026. Accessed: 2026-09-22.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 50733–50749, Vienna, Austria, 2024. PMLR. URL <https://proceedings.mlr.press/v235/wang24h.html>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Proceedings of the First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=uAjxFFing2>.
- Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering*, pp. 1482–1494. IEEE, 2023. doi: 10.1109/ICSE48619.2023.00129. URL <https://doi.org/10.1109/ICSE48619.2023.00129>.
- Chenyuan Yang, Yinlin Deng, Jiayi Yao, Yuxing Tu, Hanchi Li, and Lingming Zhang. Fuzzing automatic differentiation in deep-learning libraries. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023. doi: 10.1109/ICSE48619.2023.00105. URL <https://arxiv.org/abs/2302.04351>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. doi: 10.48550/arXiv.2405.15793. URL <https://arxiv.org/abs/2405.15793>.
- Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 283–294. ACM, 2011. doi: 10.1145/1993498.1993532. URL <https://doi.org/10.1145/1993498.1993532>.

A ADDITIONAL EXPERIMENTAL PROTOCOLS

A.1 TASK CONSTRUCTION AND ACCOUNTING

Table 5: Program collections for migration and repair, with their candidate construction, model coverage, and task counts.

Collection	Candidate construction	Model or operation coverage	Tasks
PyTorch to MindSpore	Generated translations	Operators, CNNs, MLPs, Transformers, language models	50
Saved initial translations	Five failing and five passing translations	Recurrent, convolutional, attention, and set models	10
Training-signal ablation	Controlled training faults	CNN, image MLP, Transformer, causal language model	16
JAX repair	Supplied faulty candidates	MLP and CNN	6

The main MindSpore collection contains 50 task identifiers. Twelve entries reuse existing native source programs, and the remaining entries use programs authored from public interfaces and declared semantics. The identifiers correspond to 29 distinct combinations of source program and evaluation settings drawn from 24 source files. Conditions with identical source, evaluation settings, initial candidate, method, and inputs are executed once and associated with the corresponding identifiers. Acceptance in the main table uses all 50 identifiers, while token totals count actual calls. LaDiM, SWE-agent, and MatchFixAgent share the same initial translation for each distinct input. Production edits are restricted to the candidate program, with separate temporary tests available for investigation. Sources and task specifications remain fixed.

The comparison across languages contains ten DJL application programs and eight textbook examples, averaging 221.2 nonblank source lines. LaDiM accepts one application and all eight examples. SWE-agent and MatchFixAgent each accept the same application and seven examples, while test-guided repair accepts seven examples. All repair methods preserve the initially accepted application. The saved initial translations and their generation costs are shared by the repair conditions. InterTrans uses its direct Java-to-Python path and an intermediate JavaScript path, with native search and extraction procedures. Its 2,286,034 tokens include 574,304 from incomplete generation calls and 1,711,730 from the completed search. The final evaluation compares outputs, losses, gradients, parameter updates, and observable optimizer state over two consecutive training steps.

The common MindSpore initial translations cost 429,109 tokens. This amount is included once in each repair method’s total. Subsequent LaDiM calls use 1,409,686 tokens on inputs that pass the initial checks and 3,320,339 on inputs that require repair. The corresponding MatchFixAgent totals are 5,135,863 and 6,532,453. Across the 29 distinct inputs, the median ratio of LaDiM to MatchFixAgent total tokens is 0.588. LaDiM uses fewer tokens on 19/20 initially passing inputs and 8/9 initially failing inputs, giving 27/29 overall. Seven of the nine failing inputs are repaired at the first submission. Shared initial translation costs cancel when their paired costs are subtracted. MatchFixAgent reaches full acceptance within its first external submission through its internal analysis and repair orchestration.

All failed tasks remain in each study’s declared denominator, and all failed calls remain in its cost totals. Acceptance at a submission budget scores the current candidate, including any regression after an earlier pass. An edit counts one successfully applied editing operation and can replace several code blocks across files in the declared scope. Investigation and repair share cumulative call, output, and time budgets across submissions.

Source lengths count nonblank physical lines, including comments. The 50 MindSpore identifiers use 24 source files, the signal study uses four, and the JAX study uses two. Repository lengths include all source Python files and notebook code cells, including original tests and excluding the external evaluation harness.

Table 6: Complete input and output token costs for individual migrations. Costs include initial translation and every subsequent call. Acceptance uses the declared task denominator for each collection.

Method	LLM calls	Input tokens (millions)	Output tokens (millions)	Total tokens (millions)	Accepted
<i>(a) PyTorch to MindSpore</i>					
Direct LLM	29	0.018	0.411	0.429	29/50
CodeTransEngine (Macedo et al., 2025a)	29	0.022	0.257	0.280	31/50
MSAdapter (OpenI, 2025)	–	–	–	–	15/50
SWE-agent (Yang et al., 2024)	1,097	19.845	0.987	20.831	44/50
MatchFixAgent (Ibrahimzada et al., 2026)	619	10.750	1.347	12.097	50/50
LaDiM (ours)	326	4.298	0.861	5.159	50/50
<i>(b) Java/DJL to Python/PyTorch</i>					
Direct LLM	18	0.142	0.552	0.694	1/18
InterTrans (Macedo et al., 2025b)	89	0.680	1.606	2.286	1/18
Test-guided repair	71	1.594	2.881	4.475	7/18
SWE-agent	682	23.063	1.258	24.321	8/18
MatchFixAgent	524	26.335	2.020	28.355	8/18
LaDiM (ours)	345	19.470	2.009	21.479	9/18

A.2 NUMERICAL COMPARISONS AND MIGRATION SETTINGS

The main MindSpore collection uses absolute/relative tolerances of $10^{-4}/10^{-4}$ for forward values, $10^{-4}/10^{-3}$ for gradients, and $10^{-5}/10^{-3}$ for updates. BF16 matrix multiplication uses absolute tolerance 10^{-2} . Shapes, data types, initial values, and trainability must match. Parameter and optimizer tasks test their specified enumeration, gradient assignment, or update behavior. Source and target programs begin from corresponding initial states and retain their own state over two consecutive steps for individual migration and three for repositories. The source program and task requirements are available to each method. Acceptance also checks that the target framework executes the required computations.

SWE-agent retains its native tools and controller. MatchFixAgent retains its analysis and repair orchestration through a shared DeepSeek tool backend. CodeTransEngine generates its own translation through the native direct path, and MSAdapter uses its native interface mapping. Individual migration allows 40 investigation and repair calls, 120,000 output tokens, 1,800 seconds, and four submissions. Repository limits are 80 calls, 480,000 output tokens, 3,600 seconds, and four submissions. LaDiM allows eight calls per investigation or repair stage on individual programs. Agent calls allow at most 16,384 output tokens, while initial translation and the repair loop that rewrites the complete candidate allow at most 131,072 per call. The latter has four repair rounds. Repository calls allow at most 32,768 output tokens. Output budgets include reasoning tokens, and all input usage is recorded.

A.3 RANDOM SEEDS AND INITIAL STATES

Individual migration. The main MindSpore and Java/DJL comparisons use seed 101 for public evaluation and seeds 202 and 303 for confirmation of a passing candidate. The MindSpore evaluator uses NumPy’s random generator to sample numerical inputs and training targets. Corresponding parameters are assigned identical initial values generated from a hash of each parameter name; these values remain fixed across the three input seeds. Discrete inputs, masks, and prescribed optimizer-test values retain their task-specific settings. For Java/DJL programs, each paired evaluation supplies the original initial parameters and two preprocessed batches to the candidate for consecutive training steps. The component comparisons on these programs use the same settings.

Natural faults and training signals. All ten natural translation tasks use seed 42 for public evaluation and 1042 and 2042 for confirmation. The 16 training-signal candidates use 4101, 5101, and 6101, with 4101 serving as the public seed. The paired evaluator sets the Python, NumPy, and PyTorch random states before constructing the source model and copies its initial state to both executions. A separate CPU PyTorch generator seeded with $s + 101$ produces inputs and labels for

evaluation seed s , including token inputs for text models. The signal study uses all three seeds when checking fault construction and scoring the final candidate with all signals. The additional repair collections retain their original unit probes and apply their task-specific acceptance checks without an additional multi-seed confirmation stage.

JAX repair. The MLP and CNN programs use public seed 6701 and confirmation seeds 6702 and 6703. The evaluator initializes PyTorch’s random state with the evaluation seed before constructing the source or candidate model. Inputs and class labels use a separate CPU generator with seed $s + 101$. TorchAX transfers the candidate to JAX, where native differentiation and Optax perform the checked training step.

Detection trajectories. Seeds 300, 301, and 302 define three repeated trajectories for each model. Each seed initializes the model’s floating-point parameters, and its initial state is shared across the source and target executions. Image batch order uses seed $s + 5000$, text-classification batch order uses $s + 6000$, and language-model window selection uses $s + 7000$. The fault and synchronization conditions for a given model and seed share the same initial state and 50-step batch sequence.

Time series repository. The main comparison and component ablation use public seed 101 and confirmation seeds 202 and 303. The source model is initialized with the evaluation seed, and its parameters are copied to the target. NumPy generators use seed $s + 47$ for the three training batches and seed s for the synthetic price series. The target MindSpore random state also uses s . The training script, notebook, and teaching examples are exercised in the public evaluation; confirmation repeats the numerical checks on the remaining two seeds.

Recommendation repository. The public numerical evaluation uses seed 101 for source model initialization and NumPy generation of user identifiers, features, histories, item identifiers, positions, and labels. The source parameters and retrieval corpus are then supplied to the target, whose MindSpore random state uses the same seed. The protocol assigns seeds 202 and 303 to confirmation after all public checks pass; the reported recommendation runs finish at the public evaluation. The original tests use seed 42, including Python, NumPy, and framework random states. For their exact-value history-encoder checks, the public task supplies the source parameters initialized with PyTorch seed 42. The one-epoch training command uses seed 101.

A.4 TRAINING SIGNAL STUDY

The training signal and JAX studies use the same investigation and evidence handoff procedure with editing examples and format-correction feedback in their tool prompts. The signal experiment applies four controlled modifications to four model classes. The execution fault uses an invalid optimizer keyword, the forward fault adds a constant loss offset, the gradient fault scales gradients, and the update fault changes the learning rate. Each setting retains the initial investigation, current workspace, and cumulative call budget. Its visible checks determine whether repair starts and stops. Final acceptance uses all training signals on three seeds. MindSpore measurements use the `torch4ms` forward and backward bridge, followed by optimizer updates on its parameters.

A.5 JAX REPAIR AND NATIVE CONVERSION CONTROLS

The six JAX candidates comprise an execution, forward, and gradient fault in each of an MLP and a CNN. LaDiM and Direct repair receive the same source programs and original candidates and retain their respective procedures. Each allows four submissions, 40 LLM calls, 120,000 output tokens, and 1,800 seconds, including investigation. Both methods accept every task at the first submission on seeds 6701, 6702, and 6703. LaDiM uses 76 calls and 674,009 tokens; Direct repair uses 57 calls and 353,311 tokens. TorchAX executes the candidate with JAX arrays, JAX computes gradients, and Optax applies one SGD step. Loss absolute difference, gradient norm absolute difference, and relative parameter update thresholds are 0.02, 0.05, and 0.03. The accepted repairs match the reference initial parameters exactly. The maximum final loss, gradient norm, and relative update differences are 1.192×10^{-7} , 1.510×10^{-8} , and 1.193×10^{-6} .

Ivy uses `ivy.transpile` to produce a JAX/Flax implementation of each supplied model. The `samuella/torch2jax` implementation uses `t2j` for models and tensors, followed by JAX differentiation and Optax updates. Each performs a single native conversion of the already faulty candidate. For both converters, the two execution faults retain an extra argument to a linear layer

and raise `TypeError`, leaving numerical measurements n/a. The two forward faults preserve the output scaling error and fail loss, gradient, and update checks. The two gradient faults preserve the interrupted differentiation path: loss passes, while gradients and updates fail. Each converter’s healthy MLP and CNN controls pass. Ivy spends 561.935 seconds on the six faulty candidates and 191.663 seconds on its healthy controls; `torch2jax` spends 39.234 and 15.930 seconds. Their LLM calls and tokens are zero.

A.6 DETECTION MEASUREMENTS

The detection study uses four models, three seeds, and 50 training steps. Gradient scaling and partial update suppression are evaluated both with per-step synchronization to the source state and with each execution retaining its own state. With synchronization, each target starts the checked step from the source state. Loss, gradient norm, and relative update thresholds are 0.02, 0.05, and 0.03. Figure 4 uses trajectories with separate state and reports the mean and range across 12 runs per fault. The loss threshold crossings at steps 31 and 18 refer to mean curves. Direct gradient or update checks detect all respective faults at step 1. Healthy translations pass the recorded checks, with maximum loss difference 3.815×10^{-6} .

The scalar summaries are

$$\Delta_{\text{loss}} = |L(P) - L(\hat{P})|, \quad (3)$$

$$\Delta_{\text{grad}} = \left| \|\nabla_{\theta} L(P)\|_2 - \|\nabla_{\hat{\theta}} L(\hat{P})\|_2 \right|, \quad (4)$$

$$\Delta_{\text{param}} = \frac{\|\Delta\theta - \Delta\hat{\theta}\|_2}{\|\Delta\theta\|_2 + \epsilon}. \quad (5)$$

Here $\Delta\theta$ and $\Delta\hat{\theta}$ denote source and target parameter updates, and ϵ prevents division by zero. Figure 4 divides each difference by its detection threshold. The main migration comparison uses the task-specific tensor comparisons defined above.

B EVIDENCE HANDOFF AND AGENT INTEGRATION

The component study shares the main collection, initial translations, acceptance checks, and cumulative budgets. Continuing the original conversation retains its message roles and reasoning history when the agent begins editing. Independent evidence handoff supplies the Verifier’s evidence to a separate repair conversation. Removing repair history restarts each subsequent repair attempt from the initial investigation and latest observation, while retaining the current files and cumulative budget. Removing progress reminders preserves the call and submission limits.

Table 7: Complete component and host-integration comparison on the common MindSpore collection. Token totals include initial translation and all subsequent calls.

Setting	Accepted	Total tokens (millions)
LaDiM	50/50	5.159
Continuous investigation and repair conversation	49/50	5.603
Without repair history	50/50	4.566
Without progress reminders	50/50	4.562
MatchFixAgent	50/50	12.097
MatchFixAgent with independent investigation	50/50	19.480
SWE-agent	44/50 [†]	20.831
SWE-agent with independent investigation	43/50	36.547

[†] Includes one task with unavailable final acceptance, which remains in the denominator and cost totals. Each condition uses the input reuse described in Appendix A.1.

LaDiM accepts all 50 identifiers, while the continuous conversation accepts 49. Removing repair history or progress reminders preserves full acceptance and uses fewer tokens. Twenty distinct initial translations pass the initial checks, and only two of the nine failing inputs require a second repair

submission under the complete method. Most cases therefore finish before a later repair attempt can use earlier repair history.

The component study on saved initial translations uses the same ten candidates as Table 2, including access to their supporting libraries. Its reference configuration includes examples of the edit format and corrective feedback for malformed edits. Each condition changes the named component while retaining the candidates, workspace access, acceptance checks, and repair budgets. Removing this format assistance gives the LaDiM configuration reported in the translation error comparison. Table 8 adds the progress reminder and editing assistance conditions to the three rows in the main ablation table. All conditions preserve the five initially passing programs.

Table 8: Complete component results on saved initial translations. Repair tokens cover all ten runs, including investigation and failed attempts.

Condition	Failed programs repaired	Passing programs preserved	Repair tokens (millions)	Final accepted
Independent evidence handoff	4/5	5/5	15.548	9/10
Continuous conversation	3/5	5/5	14.516	8/10
Without repair history	0/5	5/5	10.852	5/10
Without progress reminders	2/5	5/5	17.325	7/10
Without editing format assistance	4/5	5/5	13.207	9/10

The host-integration conditions prepend the same read-only investigation to SWE-agent or MatchFixAgent. Investigation calls share the host’s total call, token, and time limits. The native host retains its own history and repair policy. MatchFixAgent with investigation retains 50/50 at a higher token total. SWE-agent with investigation accepts 43/50, compared with 44/50 and one interrupted task for its native condition. Across distinct inputs, the added investigation increases SWE-agent’s repair count from seven to eight among nine initially faulty programs. It also leaves four initially accepted programs failing at the end. Table 7 records both the repair and preservation consequences in final acceptance.

C ADDITIONAL REPAIR COLLECTIONS

We also evaluate supplied faulty target programs under their respective acceptance criteria. A controlled MindSpore collection contains 50 injected faults with task-specific acceptance checks. LaDiM accepts 45/50 within four submissions, compared with 29/50 for SWE-agent and 35/50 for a direct repair agent using shared tools. LaDiM’s cumulative accepted counts are 29, 41, and 45 at one, two, and four submissions. Total token use is 39,982,844, 42,579,064, and 12,459,013 for LaDiM, SWE-agent, and direct repair, respectively. The source and target inputs required by MatchFixAgent are supplied in the paired collection below.

In a separate collection of twelve source and target repair tasks, LaDiM accepts 12/12 and MatchFixAgent accepts 11/12. They use 6,642,603 and 14,505,833 tokens, respectively. Both receive the complete source, candidate, task specification, and target library. Each repair collection uses its specified editable code scope and acceptance checks.

D NATURAL TRANSLATION REPAIR PROTOCOL

The ten supplied translations contain five initially faulty programs and five that already pass the checks. Each method receives the same source, candidate, task requirements, and supporting library. The candidate and library may be edited. The source and acceptance criteria remain fixed. Target forward computation and differentiation execute in MindSpore through `torch4ms`, followed by PyTorch optimizer updates on its parameters. The comparison uses matching inputs and initial parameters with three-seed confirmation. Loss absolute difference, gradient norm absolute difference, and relative parameter update thresholds are 0.02, 0.05, and 0.03, with an additional gradient vector L2 threshold of 0.05 and the specified layer and parameter checks.

Table 2 counts every call for each input, including failed repairs and investigation of initially accepted programs. LaDiM uses 193 calls, direct repair uses 120, SWE-agent uses 386, and MatchFixAgent uses 265. LaDiM uses 13,206,863 tokens.

The LSTM repair in Section 4.3 follows two errors through the supporting library. The first submission leaves an unregistered recurrent operator unresolved. LaDiM reproduces the failure and adds a MindSpore recurrence with its registration. The second submission exposes recursive tensor indexing; a focused test identifies the dispatch cycle, which the third submission corrects. The final public loss absolute difference is 2.38×10^{-7} , gradient vector L2 difference is 4.19×10^{-7} , and relative parameter update difference is 8.11×10^{-7} . The example uses 32 LLM calls.

E REPOSITORY PROTOCOLS AND DETAILED RESULTS

All methods receive byte-identical source repositories, common initial translations, task requirements, and acceptance procedures. The time series translation has three generation calls totaling 163,661 tokens. The recommendation translation uses 20 calls totaling 368,368 tokens. Generation proceeds from the public import structure and includes failed or truncated attempts. Each method’s reported total contains one complete copy of this generation cost together with every repair call. Table 9 separates input and output usage in repair.

Table 9: Training checks and complete costs for repository migration. Gradient and parameter update checks cover three training steps on all evaluated seeds. Total tokens include repair input and output plus common initial translation.

Method	Gradients	Parameter updates	Repair calls	Submissions	Repair input tokens	Repair output tokens	Total tokens
<i>Time series</i>							
LaDiM	9/9	9/9	35	2	2,345,473	105,096	2,614,230
SWE-agent	9/9	9/9	80	1	7,984,797	51,367	8,199,825
MatchFixAgent	9/9	9/9	69	2	13,306,003	128,114	13,597,778
<i>Recommendation</i>							
LaDiM	18/18	15/18	80	4	8,147,846	295,032	8,811,246
SWE-agent	17/18	12/18	80	1	7,889,936	58,248	8,316,552
MatchFixAgent	18/18	16/18	80	1	8,215,327	95,385	8,679,080

The time series protocol compares the core model for three consecutive Adam steps from corresponding source parameters, with dropout disabled and input shape $4 \times 20 \times 1$. Preprocessing and prediction absolute tolerances are 10^{-9} and 10^{-4} . Loss absolute difference, gradient vector L2 difference, and relative parameter update tolerances are 0.02, 0.05, and 0.03. Learning rate behavior is checked around its schedule boundaries. The public evaluation executes the 100-epoch training workflow, notebook cells, and teaching snippets in their source order. Each entry point must produce the required predictions and plots and execute native differentiation. The notebook and snippets provide execution coverage, while the core model supplies paired numerical training measurements. All three methods pass 23 behavior checks per seed, giving 69/69 across the public and two confirmation seeds.

The recommendation protocol covers six model paths that execute in the source framework with three sequential Adam updates. Source exceptions and empty returns are checked against the corresponding source behavior. Both the numerical worker and training command invoke the candidate’s optimizer factory with learning rate 0.001. The source Adam settings use decay coefficients 0.9 and 0.999 and epsilon 10^{-8} . Representation absolute tolerance is 10^{-4} . Loss absolute/relative tolerances are 0.002/ 10^{-4} , gradient tolerances are 10^{-4} /0.005, and relative parameter update tolerance is 0.03. The evaluation includes all ten original tests and one epoch of the training command on 16 synthetic examples.

Table 10 reports the declared checks on the public seed. The 114 numerical comparisons comprise 108 checks over six paths, three steps, and six training signals, plus six inference retrieval checks. Twenty-four execution, step count, and native differentiation checks and seven source behavior checks bring the behavior check total to 145. SWE-agent and MatchFixAgent each lack one

Table 10: Recommendation repository checks passed on the public seed. The numerical checks are a subset of behavior checks, and the six training signal rows break down the training checks. One unmeasured retrieval check remains in each baseline’s denominator.

Check	LaDiM	SWE-agent	MatchFixAgent
Behavior checks passed	130/145	104/145	129/145
Numerical checks	99/114	74/114	99/114
Training checks	93/108	69/108	94/108
Inference retrieval	6/6	5/6	5/6
User representations	18/18	16/18	18/18
Item representations	6/18	6/18	6/18
Forward loss	18/18	18/18	18/18
Training return loss	18/18	0/18	18/18
Gradients	18/18	17/18	18/18
Parameter updates	15/18	12/18	16/18
Original tests	10/10	10/10	0/10
Training command	1/1	1/1	1/1
File coverage	1/1	1/1	0/1
Documentation and dependencies	1/1	1/1	0/1

retrieval measurement after reward model inference fails. That entry remains n/a in both the 114 and 145 denominators. MatchFixAgent passes 0/10 original tests: an unmigrated PyTorch import in its history-encoder test prevents collection under the native MindSpore requirement. No method reaches the two confirmation seeds on this repository.

LaDiM passes all 18 loss and 18 gradient checks and 15/18 parameter update checks in the main recommendation comparison. Later item embeddings diverge in all six model paths at steps two and three. These failures remain alongside the successful reward model inference, retrieval, training command, and ten original tests.

F REPOSITORY CONTEXT AND EVIDENCE

The repository extension separates the version of its dependency plan from the version of the candidate files. Every recorded test stores the candidate version on which it ran. An actual change to candidate code makes earlier measurements stale, and a new result replaces the preceding result for the same test even when the new test fails. Work-unit checkpoints additionally depend on their declared file scopes, test selections, and prerequisite units. A checkpoint without a selected test records syntax validation only. The final acceptance decision uses the complete repository evaluation.

Full tool observations, file reads, and conversations are archived outside the editable workspace. Records carry content hashes and can be retrieved by identifier, JSON pointer, and character range. When the context is rebuilt, it retains the current dependency plan and generation status, the complete structured hypotheses and next steps, the latest measurements, and the last tool batch. Cached code consists only of content returned by earlier reads, with matching current file hashes and relevance to the active unit. The cache contains at most 120,000 characters. Reconstruction is triggered by an independent role handoff, an actual work-unit switch, or messages exceeding 75% of the configured context capacity. Ordinary stage continuation and the first selection of a work unit retain the existing conversation.

For the repository experiments, diagnosis allows up to six calls and 64,000 output tokens. The remaining repair calls and output budget are allocated over the remaining external submissions, with at most 16 calls in the first repair stage. The cumulative limits apply to the whole repository.

G REPOSITORY COMPONENT RESULTS

G.1 CUMULATIVE COMPONENTS

We add Verifier investigation, independent evidence handoff, and repository context management cumulatively to the Repair Agent on both repositories (Table 11). Each condition runs once with the same source, initial candidate, evaluation inputs, seeds, thresholds, full training signals, and total budgets. The base condition retains the repair prompts, ordinary inspection and editing tools, syntax checks, and progress reminders. Investigation continues in the repair conversation until independent evidence handoff is added. All four conditions accept the time series repository and pass 69/69 checks; all finish at 131/145 on recommendation, giving one fully accepted repository out of two per condition.

Table 11: Cumulative agent components on both repositories. Calls cover investigation and repair; end-to-end tokens also include the shared initial translation. Each row adds to the preceding row within its repository.

Condition	Time series			Recommendation		
	Behavior checks passed	Calls	End-to-end tokens	Behavior checks passed	Calls	End-to-end tokens
Repair Agent	69/69	26	6,128,331	131/145	62	8,335,687
+ Verifier investigation	69/69	18	3,896,158	131/145	59	8,912,546
+ Independent evidence handoff	69/69	26	6,047,074	131/145	68	9,556,058
+ Repository context management	69/69	22	1,936,579	131/145	80	10,191,440

On time series, the complete context mechanism uses 1,936,579 end-to-end tokens, 68.4% fewer than the base Repair Agent. The paired comparison of repository context management in the main text uses the two conditions that already include investigation and independent evidence handoff, reducing tokens from 6,047,074 to 1,936,579. Investigation and repair input tokens fall from 5,730,588 to 1,699,639. The two truncated model responses in the condition without repository context remain in its call and token totals.

On recommendation, adding the components increases token use while all four conditions retain 131/145 checks and pass all ten original tests. The 14 unresolved checks comprise item embeddings in six model paths at steps two and three, and two parameter updates. The first three conditions reach the context capacity after 62, 59, and 68 calls. The complete method rebuilds context six times and continues to the 80-call limit, with the same final coverage. Context management sustains further repair in this task, while the additional calls leave the remaining training differences unresolved.

G.2 REPOSITORY STRUCTURAL ANALYSIS

The independent structural analysis comparison retains Repair Dependency Graph Planning, notebook tools, ordinary file inspection, independent evidence handoff, and evidence management in both conditions. The condition without Repository Structural Analysis removes the tool’s file, import, symbol, and notebook cell inventory, together with the automatic list of unassigned files. Its map tool returns the existing plan state. The complete method obtains structural information when the agent calls the map tool; initialization and context reconstruction do not perform the scan. Each condition runs once on each repository with the same initial translation and repository budgets.

Repository Structural Analysis reduces time series calls from 42 to 30 and end-to-end tokens by 19.8%, with both conditions completing all 69 checks after two submissions. On recommendation, both finish at 131/145, and retaining structural analysis reaches this coverage at the third submission instead of the fourth. Its 9,961,503 end-to-end tokens are 38.9% higher than the 7,172,218 used without structural analysis. The two conditions retain the same final 14 failures described above. The complete references in this independent study and the cumulative study are separate runs, with their costs retained in the corresponding tables.

Table 12: Independent repository component ablations. Both studies share the complete method shown for each repository. Calls cover investigation and repair; end-to-end tokens include initial translation.

Condition	Time series			Recommendation		
	Behavior checks passed	Calls	End-to-end tokens	Behavior checks passed	Calls	End-to-end tokens
Complete method	69/69	30	2,126,900	131/145	80	9,961,503
Without Repository Structural Analysis	69/69	42	2,651,248	131/145	80	7,172,218
Without Repair Dependency Graph Planning	69/69	21	1,803,551	130/145	69	9,213,723

G.3 REPAIR DEPENDENCY GRAPH PLANNING

The planning ablation removes the work-unit plan, prerequisite readiness checks, unit file scopes, checkpoint propagation through dependencies, and context reconstruction when the active unit changes. It retains Repository Structural Analysis, ordinary file and notebook tools, edit transactions and syntax checks, independent evidence handoff, evidence associated with code versions, and reconstruction at the context capacity. The entire permitted target repository remains editable. The complete references in Table 12 use the same source, initial candidate, inputs, seeds, thresholds, method implementation, and budgets as the conditions without planning. Each condition runs once; the complete references are shared with the structural analysis comparison.

Without Repair Dependency Graph Planning, time series completes all 69 checks after one submission, using 21 calls and 1,803,551 end-to-end tokens. This is 15.2% fewer tokens than the complete method. On recommendation, removing planning uses 7.5% fewer tokens but finishes at 130/145, one check below the complete method. The additional failure is the position model’s parameter update at step three. Both conditions fully accept one of the two repositories.

The recommendation trajectories show how this final difference develops. Without planning, successive submissions pass 128, 103, 133, and 130 checks. The second-stage change to a shared training entry point causes the first regression, and the final submission regresses from the intermediate 133 checks. The complete method’s corresponding trajectory is n/a, 122, 131, and 131. Its first submission returns eight execution failure flags outside the 145 protocol checks, leaving all protocol measurements unavailable. The first submission without planning and the second complete-method submission each lack one retrieval measurement; both final candidates have all 145 measurements. Removing planning exhausts the 480,000-output-token budget after 69 calls, while the complete method uses all 80 calls. The final scores retain these later regressions and all associated costs.

H NATIVE JAX MIGRATION WITH CONSECUTIVE TRAINING STEPS

This study uses twelve public model workloads from PyTorch examples, minGPT, and torchvision. They cover image classification, super-resolution, a variational autoencoder, image generation, recurrent and Transformer language models, time series, two policy learning models, graph attention, GPT, and ResNet. Each workload executes three optimization steps. The shared initial translation produces ten nonempty candidates: eight pass the three-seed evaluation, and two fail. The image generator and Transformer language model exhaust their initial output budget without returning code. The failed nonempty candidates, a time series model and ResNet, form the common repair collection for all four methods.

Candidates implement the forward computation and explicit buffer state in native JAX. The evaluator independently invokes JAX differentiation and the specified Optax optimizer for three consecutive steps on seeds 8101, 8102, and 8103. Source parameters, buffers, and batches are supplied to each method. Initial states match exactly, and full outputs, losses, gradients, updates, parameters, and buffers are compared elementwise in float64 with absolute/relative tolerances $10^{-7}/10^{-5}$. Tensor names, shapes, and data types must also match. The optimizer executes in the evaluator. Each repair condition has at most 40 calls, four submissions, 120,000 output tokens, and 1,800 seconds; each initial translation has one call with at most 16,384 output tokens.

Table 13: Repair of two initially failing native JAX translations from twelve source workloads. End-to-end tokens for the two repair tasks include 31,427 shared translation tokens; costs for all twelve sources include all 139,724 translation tokens. All four methods accept 1/2 repair tasks and 9/12 sources after repair.

Method	Repair calls	Repair tokens	Two repair tasks end-to-end tokens	All twelve sources end-to-end tokens
LaDiM	35	1,840,066	1,871,493	1,979,790
Direct repair	28	760,935	792,362	900,659
MatchFixAgent	23	258,914	290,341	398,638
SWE-agent	66	1,013,532	1,044,959	1,153,256

All four methods repair the time series candidate’s input slicing and output concatenation shapes and pass all three steps on all three seeds. LaDiM, MatchFixAgent, and SWE-agent pass at their first submission, and Direct repair passes at its second. LaDiM, Direct repair, and SWE-agent also turn the truncated ResNet candidate into executable code. All first-step arrays pass, but later steps contain gradient and update differences, followed by further output and buffer differences. MatchFixAgent terminates when its native analysis parses the truncated candidate and encounters a syntax error; its numerical results are n/a, and its four completed calls remain in the costs. All four methods therefore finish at 1/2 repair acceptance.

The eight initially passing sources are CNN classification, super-resolution, the variational autoencoder, the recurrent language model, both policy learning models, graph attention, and GPT. Adding the repaired time series model gives 9/12 accepted sources for every method. The complete initial translation ledger contains 29,455 input and 110,269 output tokens, totaling 139,724, including both generation failures. Table 13 distinguishes this source-level cost from the 31,427 translation tokens for the two repair tasks. The ResNet results show why consecutive training checks remain necessary after the first step agrees; the final three-seed evaluation retains the later discrepancies.